

1. OOP**02**

如果你没有专门学习过面向对象编程，可针对以下关键词查阅资料，重点**自学**：

对象/属性/方法、类/构造/析构、继承/派生、重载、重写、虚方法、模板

2. STL Vector**02A2**

- 试仿照讲义中的示例，学习使用STL中的vector，并逐步了解它的更多接口，乃至更多的ADT；
- 试设计若干计算任务，对vector的计算效果做一测试。

3. DSA Vector**02A3**

- 在实例代码包中找到Vector对应的项目，编译并运行对应的测试程序；
- 阅读主测试程序的代码，了解测试的**逻辑与过程**；
- 对照测试的每个环节，阅读对应算法的代码，进一步理解测试所报告的结果。

4. Batch & Single**02B1**

在讲义中，我们选择了先实现向量**区间**的插入与删除接口，然后再借用它来实现**单元素**的插入与删除接口。我们为何不倾向于反过来，也就是说：先实现后者，再通过反复地调用它来实现前者？

5. memcpy() & memmove()**02B1**

在insert(r,A,lo,hi)及remove(lo,hi)中，时间成本主要消耗于**后缀**的移动。

- 示例代码是通过显式的循环来完成移动的，如果颠倒循环的方向，可能会有什么问题？何时没问题？
- 如果改为调用memcpy()或memmove()来实现上述移动，需要注意什么方面？

6. Worst Case Of remove()**02B1**

- 试确认：remove(lo,hi)消耗的时间为 $\mathcal{O}(n - hi)$ ，仅取决于**后缀**[hi,n)的长度，hi越小复杂度越高；特别地，当hi = 0时为**最坏情况**，耗时 $\mathcal{O}(n)$ 。
- 试确认：当hi = 0时，其实不会删除任何元素，后缀的前移乃是无意义的**原地踏步**。
- 针对这一现象，你觉得应该如何**改进**？
- 就总体而言，你的改进是否**值得**？

7. Amortization**02B[2+3]**

讲义中证明过，若insert()操作仅限于**单元素**插入，则expand()的均摊时间复杂度为 $\mathcal{O}(1)$ 。

- 试证明：只要是按几何级数来实现expand()算法，无论选用多大的**常倍数**，都可保证均摊 $\mathcal{O}(1)$ ；
- 这里的常倍数（2、3、4甚至更大），对**时间复杂度**的常系数有何影响？
- 讲义中用装填因子来衡量向量的**空间利用率**，上述常倍数对这方面有何影响？
- 试查阅资料以了解，STL等**实用环境**中的向量是如何扩容的？如果也是几何递增方式，常倍数选作多少？
- 若允许通过insert()**批量**地插入元素，expand()的均摊时间复杂度依然是 $\mathcal{O}(1)$ 吗？

8. Amortization**02B[2+3]**

我们知道，对expand()的调用可分为两种情况：**无需**扩容时直接返回，以及**真地**进行了扩容。我们也知道，“expand()均摊时间复杂度为 $\mathcal{O}(1)$ ”的结论，是相对于对expand()的**所有**调用来做均摊而得的。那么，为什么不能仅相对于**后一类**情况来做均摊？

9. shrink()**02B[2+3]**

- a) 对照讲义的分析，阅读并理解expand()算法；
- b) 进一步阅读和理解shrink()算法（尽管讲义中并未对它做更多讲解）；
- c) shrink算法为何将**装填因子**的阈值设为1/4，而不是与expand算法**相对应**地取作1/2？

10. find()**02B4**

试跟踪执行过程以验证：

- a) 在查找失败时，find()中的循环会正确终止，并返回 $hi = lo - 1$ ——调用者据此可以确认失败；
- b) 这种情况下的最后一步循环，条件测试中的“ $e \neq _elem[hi]$ ”会不会**越界**（尤其是当 lo 为0时）？

11. find() Using Sentinel**02B4**

find(e, lo, hi)算法中的while循环，每一步迭代都会涉及两次比较：一次是在**秩与秩**之间($lo < hi--$)，另一次是在**元素与元素**之间($e \neq _elem[hi]$)。实际上，可以通过增加**1次**后一类比较，省略掉**所有**的前一类比较。具体思路是，在进入循环之前先备份 $_elem[lo-1]$ ，然后将e的副本放入 $_elem[lo-1]$ （作为**哨兵**），待退出循环后再予复原——这样，同样能保证返回 $lo-1$ 当且仅当查找失败。

- a) 为此实现这一思路，向量结构的总体设计还有哪些方面需要调整？
- b) 试实现这一新方法，并通过测试验证实际的改进效果。

12. find() + dedup()**02B4**

我们知道，find(e, lo, hi)算法约定查找失败时返回 $lo-1$ ，而dedup()算法调用find()时总是 $lo = 0$ 。

- a) 此时如果查找失败，作为Rank/unsigned int的 $hi = lo-1$ 会是什么？
- b) 此时dedup()将该返回值与-1做比对，是否会出现错误？

13. dedup()**02B4**

考查本节介绍的dedup()算法。

- a) 试确认，若向量中的所有元素**互异**，则需运行 $\mathcal{O}(n^2)$ 时间；
- b) 试确认，若向量中的所有元素**全等**，也需运行 $\mathcal{O}(n^2)$ 时间；
- c) 试改进该算法，在不改变**减而治之**的整体策略与逻辑，仍通过调用find()来检测重复的前提下，使得无论如何，消耗于remove()的时间累计不超过 $\mathcal{O}(n)$ 。
- d) 如此改进之后，尽管消耗于find()的时间累计仍需 $\mathcal{O}(n^2)$ ，但**实际**的运行速度为何会有极大提高？
- e) 如何**实质地**提高该算法的效率呢？
- f) 可否进一步优化dedup()算法，使得即便在最坏情况下，其复杂度也会严格地低于 $\mathcal{O}(n^2)$ ？如果可以，试给出具体算法；否则，试说明理由。

14. Function Object**02B5**

- a) 针对函数对象Increase，阅读示例代码包Vector项目中对应的代码，理解**函数对象**的形式与机制；
- b) 试仿照示例代码中的形式编写更多的函数对象，实现对向量**不同功能**的遍历（比如减一、加倍、求和等）。

15. 判等器+比较器**02C1**

本节uniquify()的两个版本虽然都是针对有序向量，却依然使用了“==”和“!=”来做元素的比对。也就

是说，这里假定：元素的类型要么**天然**就支持这样的比对，要么已对这些操作符做过**重载**。

如果你还不了解如何通过重载以实现这样的效果，可通过查阅一些相关资料来自学。

16. 勤奋版uniquify()

02C1

该版本一旦检测出相邻的逆序对，便会迫不及待地实施remove()，由此导致总体效率过低。此外，你或许还会担心，倘若某次remove()触发了shrink()缩容操作，从而使内部的数据区_elem[]迁往**新址**。那么，该算法接下来还能不能准确地访问向量的各个元素？当然，答案是**能**。试就此自行核对。

17. 懒惰版uniquify()

02C1

通过简洁而精巧的设计，该版本可在 $O(n)$ 时间内完成**有序**向量的去重操作，且空间成本仅为 $O(1)$ 。

- a) 试确认：在遇到**第一个**重复元素之前，循环中的移动操作都是**徒劳无益**的——不过是自己到自己的赋值；
- b) 针对这一问题，你有什么**改进**的思路？试实现之。
- c) 这种改进是否值得？

18. Stability

02[B4+C1]

无论是dedup()算法还是（勤奋版、懒惰版）uniquify()算法，每当发现一对相等元素，它们总是倾向于删掉**居后者**。如此，去重的整体效果将是，在每一组互等的元素中只有**最靠前者**能够幸存下来。

然而作为一个**稳定**的算法，本来应该是保留**最靠后者**的。如何调整这些算法，使之具有**稳定性**呢？

19. Priority By Probability

02C2

binSearch()版本A处理三个分支的次序是“小于 ~ 大于 ~ 相等”，当然，应该还有5种可能的次序。

- a) 试验证：就渐进复杂度而言，它们并无实质区别。
- b) 试解释：就常数系数而言，它们还是有很大的区别。
- c) 上述区别主要决定于哪些因素？
- d) 试以32位无符号整数构成的、长度为 10^6 的有序序列为例，估算出上述区别。

20. Comparator

02C2

讲义指出，binSearch()版本A的时间成本主要取决于，在while循环中 $S[mi]$ 与目标e的**比较**操作次数；并进而由此说明，该算法的不足在于向左、向右的转向耗时不平均，大致是**1:2**的比例。

然而我们从代码可以看到，明明还有其它的**比较**操作及算术运算，且总共也有 $O(\log n)$ 次。

- a) 为何这部分成本被“无视”了？
- b) 为何左转、右转的时间成本不是**2:3**、**3:4**、**4:5**...？

提示：参看新版教材的脚注

21. Overflow

02C2

为了避免运算**溢出**，你可能会将binSearch()算法对轴点的计算方式改为“ $mi = (hi-lo)/2 + lo$ ”。

- a) 试确认，这样的确可以避免数位溢出；
- b) 对于binSearch()**适用**的查找规模而言，是否有**必要**如此调整？

22. Duplicates 02C2

不难发现，当有**多个**元素与查找目标相等时，`binSearch()`版本A只是“随机”地返回了其一。

试确认：此时具体返回何者，取决于查找区间的**宽度**，以及相等元素的**数目**。

23. Return value 02C3

- 试证明：`binSearch()`版本B为何首先需要特判“ $e < S[lo]$ ”的情况？
- 此前的版本A为何不需要如此特判？

24. Empty Sequence 02C4

- 如果查找目标 $e < S[lo]$ ，`binSearch()`版本C会如何处置？
- 如果查找区间的宽度 $hi - lo = 0$ 呢？
- 在上述情况下，版本A和版本B又会如何处置？

25. `binSearch()`: Version C: Correctness 02C4

本节通过建立循环不变性，证明了该算法的正确性。你还有其他证明方法吗？

26. `binSearch()`: Version C: Average Search Length 02C4

本节指出，该算法总是满足： $ASL_{succ} = ASL_{fail} = \mathcal{O}(1.00 \cdot \log n)$

- 试证明，这一结论对于任何 $n = 2^k - 1$ 都成立；
- 试证明，这一结论对任何 $n > 0$ 都成立。

27. `binSearch()` 02C[2~4]

考查二分查找的各种版本。

- 它们都以搜索区间收缩至1甚至0作为迭代退出的条件（对应于递归版本，即相当于递归基）。试分别验证，无论如何迭代退出时，它们**最终**的区间宽度的确为1或0。
- 试证明：无论何种算法，失败情况总是比成功情况多出一**种**。
一次查找过程中所做的元素**比较**、**比对**操作的总次数，称作**查找长度**（Search Length, SL）；多种情况下的期望值，也称作**平均查找长度**（Average Search Length, ASL）。
- 试针对长度为 $n = 2^k - 1$ 的向量，估算出个算法在**成功**情况下的**平均查找长度**。
- 试针对长度为 $n = 2^k - 1$ 的向量，估算出个算法在**失败**情况下的**平均查找长度**。

28. Loop Termination 02C[2~4]

我们看到，不论`binSearch`的三个版本还是`fibSearch`，算法的主体都是一个**while**循环。试逐个地确认，它们的确都能**终止**，且在循环的出口处`lo`和`hi`的数值都正如我们所**预期**。

比如，以`binSearch`版本C为例，你需要验证：对于输入时合法的任意`lo`和`hi`，尽管**while**循环的持续条件是“ $lo < hi$ ”，但在出口处**必有**“ $lo == hi$ ”，而**不致**出现“ $lo > hi$ ”的情况。

29. `intSearch()` 02C5

即便是有序序列，数据的分布也可能很**畸形**。试举出这样的实例，导致插值查找算法的复杂度反而过高。

30. `intSearch()` 02C5

本节针对均匀分布的数据，给出了轴点的估算公式。如果换成**其它**的分布形式，比如**高斯**分布呢？

31. Interpolation/Binary/Sequential Search**02C[2~5]**

课上指出，**长度**不同的查找表所适用的查找算法也不同。大体来说，随着查找表长度的递减，应当以一种**接力**的方式来实现查找，即依次采用插值查找、二分查找、顺序查找。当然，这只是一个**定性**的趋势和原则，应当结合具体的应用与硬件平台来确定它们之间**准确**的分界线。

- 试在你的电脑，设计实验并通过实验数据确认，何时应当从二分查找**切换**至顺序查找；
- 如果有可能，可针对更多平台及应用数据进行实验，得出对应的结果。

32. bubblesort()提前终止版**02D**

试在示例代码包中找到该算法对应的项目：

- 适当修改测试程序并自行设计测例，了解其中**sorted**标志的演化过程，验证该算法的**逻辑**正确无误；
- 对足够大的数据做实测，估计出这一改进的平均效果（比如，**平均**能节省多少趟扫描）。

33. bubblesort()跳跃版**02D**

试在示例代码包中找到该算法对应的项目：

- 适当修改测试程序并自行设计测例，了解其中**last**标志的演化过程，验证该算法的**逻辑**正确无误；
- 对足够大的数据做实测，估计出这一改进的平均效果（比如，**平均**能节省多少趟扫描）。

34. mergesort()**02E**

如果选用**另一**常数 $\lambda \neq 1/2$ ，总是按照 $\lambda : 1 - \lambda$ 的比例来切分子序列，归并排序的渐近复杂度将是多少？

35. mergesort() ~ merge()**02E**

该算法的实质计算无非是二路归并，讲义及示例代码中该算法的实现虽然表面上很简洁，但由于反复地申请（new）和释放（delete）空间，时间复杂度的**常系数**其实颇高。

- 你认为可以如何减少这类**动态**内存操作，以降低复杂度的常系数？
- 试编程实现你的办法，并做实测对比。

36. merge()**02E**

示例代码中实现的二路归并算法，似乎只处理了**A先**耗尽的情况，**B先**耗尽呢？

37. merge()**02E**

我们注意到，即便待归并的两个序列已经**完全有序**，示例代码中实现的merge()仍是墨守成规，花费线性的时间以完成“归并”。

- 试改进merge()算法，使之能在上述情况下及时返回；
- 为此你的merge()需要多花多少时间？
- 如此多出的时间，相对于所得**收益**孰大孰小？

38. In-place Mergesort In STL**02E**

STL中的归并排序对merge()算法做了调整，从而彻底避免使用更多的辅助空间。

- 试查阅相关资料，理解其原理及实现细节；
- 这种调整付出了什么**代价**？
- 相对于空间方面的**收益**，这些代价是大是小？为什么？

39. Bitmap**02F1**

- a) 试在示例代码包中找到该算法对应的项目，着重阅读并理解其中利用**位运算** (bitwise operation) 实现比特定位的原理及过程；
- b) 对照02B1中的expand()算法，阅读Bitmap::expand()的代码并理解其实现原理；
- c) 为保证常数时间内完成Bitmap的整体复位，02G3节介绍了Hopcroft的**校验环**技巧。试从算法演示包中找到对应的Excel版演示，通过实践体会和理解其原理及过程；
- d) 如果只是着眼于在常数时间内整体复位，其实还有其它方法，你能想出来一种吗？
- e) 相对于Hopcroft的方法，你的方法有何优势和劣势？
- f) 自学C++模板类bitset，并从接口于功能的角度，与课上讲授的Bitmap做一对比。

40. Vector<int>::dedup() With A Bitmap**02F2**

- a) 试借助Bitmap，实现一个对整数向量的去重新算法；
- b) 设整数的取值范围为 $[0, m)$ ，向量长度为 n ，你的新算法的时间、空间复杂度各是多少？
- c) 试验证，你的算法不仅能够实现去重，同时也能完成**排序**。

41. Bitmap::clear() By Rotate Or Shift**02F3**

在Hopcroft改进的Bitmap中，清除一个元素对 $F[]$ 数组而言直截了当，但对 $T[]$ 来说却比较复杂。我们在课上看到，Hopcroft以一种**旋转**的方式，将末元素**前置并顶替**被删除者，从而在 $O(1)$ 时间内实现了删除。

- a) 试按照常规的后缀**前移**的思路来编程实现Bitmap::clear()；
- b) 这种实现方式的复杂度如何？包括程序的运行时间，以及编写和调试程序所消耗的时间；
- c) 对比两种思路，反过来体会Hopcroft技巧的高明之处。

42. Vector<int>::dedup() With Hopcroft's Bitmap**02F3**

- a) 试说明，当整数的**取值范围**很大 ($m > n$ 甚至 $m \gg n$) 时，借助Hopcroft的Bitmap可将向量去重算法的效率从 $O(n + m)$ 优化到 $O(n)$ ，也就是说运行时间与取值范围无关（尽管仍与空间复杂度相关）。
- b) 相对于使用常规的Bitmap，作为补偿，这种方法有何**倒退**？